

NovodeX Viewer Documentation

1 Introduction

The NovodeX Viewer is an application that loads a physics scene stored in NovodeX' .pds.ods file format, and displays it. The user may then interact with the scene as it is simulated by the NovodeX physics technology. The source code of the viewer is freely available. This documentation describes the use of the viewer, and the respective file format. Users of the SDKs may either reuse our file format for loading physics scenes into their respective applications, or use their own proprietary file format. In the former case, the source code of the viewer that reads the file format may be reused. Our file format is currently used by all of our demos, and the 3D Studio MAX and MilkShape 3D exporters that come along with NovodeX Personal Edition. More 3rd party exporter tools are under development.

2 Viewer

2.1 Executable Versions

The Viewer application executable is available in two different versions. The standard version can be used without restrictions, while the personal version only runs on a system with a personal license. The personal version can be recompiled using the source code included in the personal edition. The executables have the following path relative to the NovodeX home directory:

Bin/win32/NovodeX.exe	Standard executable for Windows
Bin/win32/NovodeXPersonal.exe	Personal executable for Windows (Only runs on a licensed system)

2.2 Source Code

The source code of the NovodeX Viewer and related files are located in directories relative to the NovodeX home directory:

Viewer/src/	Source
Viewer/compiler/vc6/	Microsoft Visual Studio 6.0 workspace and project files
Viewer/doc/	This Documentation

2.3 Source Code

The source code of the NovodeX Viewer and related files are located in directories relative to your NovodeX home directory:

Viewer/src/	Source
Viewer/compiler/vc6/	Microsoft Visual Studio 6.0 workspace and project files
Viewer/doc/	This Documentation

2.4 Command Line and Startup

The path to a .pds.ods file may be specified as a command line argument to the viewer. If no file is specified, a file open dialog box will pop up and prompt for a file. However, by far the fastest way to open files is to place a shortcut to the viewer on your desktop, and drag .pds.ods file icons on it from the Windows explorer.

The SDK includes demo scenes which may be opened in the directory *ViewerScenes*.

2.5 Full Screen Mode

It is possible to run the demo in full screen mode. To do this, open the file app.gla.ods in the demo directory, and uncomment the line with glutGameModeString. If this file doesn't yet exist, you may create one, or copy it from the directory of one of the demos provided.

2.6 Controls

Once you start a scene, the following controls are available:

h	Displays help text.
escape	Quits the viewer.
k	Dumps the camera settings to the console. The position and angles that are specified may be pasted into the camera block of scene files to determine the starting camera settings.

2.6.1 Flight and Interaction

To look around, press and hold left mouse button in view port while moving mouse.

w or up arrow	Fly forward
a or left arrow	Fly left
s or down arrow	Fly backward
d or right arrow	Fly right
e	Fly up
c	Fly down

When the simulation is not paused, you may click and drag objects with the right mouse button to move them. The amount of force the mouse will apply depends on the settings in the scene file that you loaded, so it may be that some objects are too heavy to be moved. Also, objects may be fixed so that they are immovable. While dragging an object, you may delete it by pressing **delete**.

Some scenes may have special active objects in them which, when clicked with the right mouse button become activated and perform a special function. An example is the truck in the monster truck scene. Here clicking on the car chassis will switch to a driving mode. The truck may then be driven using the **arrow keys**. An overturned vehicle can be reset with '**r**'. Press **escape** to leave the car and return to fly mode.

2.6.2 Scenes

A .pds.ods file may contain one or more scenes.

n Cycles through the scenes. If there is only one scene, it will restart it.

2.6.3 Simulation

1 Pauses the simulation. You may still fly around, but naturally you may not interact with the scene.
2 Advances the simulation by a single frame, then pauses.
3 Runs the simulation in real time. (default)
5 Runs the simulation as fast as your machine will go.

2.6.4 Debug Visualization

Most options are available through pressing the mouse middle button on the Viewer window. Use 'q', 'z' to scale debug visualization elements.

ViewerVisualize

wireframe	Toggles wire frame rendering
hide	Toggles mesh rendering.

SDKVisualize

World Axes	World coordinates axes.
Actor Axes	Actor object coordinates axes.
Body Axes	Body axes, with origin at the center of mass.
Body Inertia	Visualization of the inertia tensor as a box.
Body L.Velocity	Linear velocity of the body (white arrow.)
Body A.Velocity	Angular velocity of the body (black arrow.)
Body Reduced	Reduced joint link hierarchy. (link: green cross, root link red cross.)
Body JointGroups	Joint link hierarchy. (root link: red cross.)
Body Sleep	Changes the color of the body inertia visualization to black, if a body goes to sleep.
Joint World Axes	Joint world axes, located at ideal joint position.
Joint Local Axes	Joint local axes, located at body joint position.
Joint Limits	Joint limits, turning yellow, if joint position hits the limit.
Contact Points	Contact points between actors.
Contact Normals	Normals direction of contacts.
Contact Forces	Repulsive contact forces.
Contact Error	Penetration vector.
Collision Shapes	Shapes used for actor collision.
Collision AABBs	Axes aligned bounding box of actors, used for collision detection.

Profiler (Copyright (c) 2002 Jonathan Blow)

none	Toggle profiler.
self	Times of different tasks without the times of sub tasks.
self-stddev	Standard deviation of "self"
hier	Times of different tasks including times of their subtasks.
hier-stddev	Standard deviation of "hier"
toggle pause	Stops and Starts the profiler.

Some options can be accessed directly with the keyboard.

b	Shows bounding volumes used for collision detection.
x	Shows contact points (blue).
q, z	Increase and decrease the size of debug visualization objects.
0	Activate Real Time Profiler (Copyright (c) 2002 Jonathan Blow).
8, *, 9, (	Change profiler display modes.
)	Freeze profiler display.

3 File Format

3.1 ODS files

ODS stands for “Object Description Script”. It is a very simple C-like hierarchic syntax to describe data structures. It does not itself specify any semantics (meaning) for this data. In this regard it is similar to XML. However, ODS files are much more human readable.

The following notation describes the recursive structure of ODS files:

ODScript	= Block
Statement	= Block Terminal
Block	= indent '{' {Statement} '}'
Terminal	= indent ';' ;

In other words, ODS file always contains a single *block*. A block starts with an *identifier* string. It is followed by an opening brace, one or more *statements*, and a closing brace. A statement is either a block or a so-called *terminal*. A terminal is simply an identifier followed by a semicolon rather than braces.

An identifier may be any alphanumeric string. Those which consist of several words or contain characters with special meaning to the ODS parser ({, }, ", #, /) should be put in quotation marks. At the moment the parser can deal with multiword quote less identifiers, but this should not be depended on.

ODS files are case sensitive, but white space does not matter.

Comments in ODS scripts may be specified in two ways:

A hash mark (#) starts a line comment – everything until the next line break is ignored.

C style multiline comments (/* */) also work.

ODS files may be scrambled to prevent modification. See the header file ODBlock.h which is distributed with the graphics library for details.

We have followed the convention of using a double file extension to identify ODS files. The code after the last period in the file name is “ODS” and identifies it as a kind of text file with the above properties. Another code immediately preceding this identifies the kind of information stored in the file. For example, the file with the name:

wood.mat.ods

is assumed to be an ODS file with material information that is understood by our graphics engine. This and other such ODS data types used by the viewer are described below.

3.2 PDS.ODS files

PDS stands for Physics Demo Script. An annotated example is given below. Again, the explanation comments are in italics. Non-keyword variables are bold.

```
PDS  #code identifying the format
    {
    Acts
        {
        /*
        A list of act (a.k.a scene) files follow
        The format of these files is given in
        the next section.
        */
        firstScene.psc.ods;
        scene2.psc.ods;
        lastScene.psc.ods;
        }
    Models
        {
        /*
        Models are the graphical meshes, along with materials, which are used to
        display a body in the view port. They have nothing to do with physics, and
        will be replaced by custom solutions in a real game. A list of models are
        assigned to names here.
        */
        BallModel {ball.mod.ods;}
        EraserModel {eraser.mod.ods;}
        }
    Actors
```

```

{
/*
The Actors block contains a list of actors.
An actor is the combination of a graphical model with collision
properties. The actor is only a template in the sense that several instances
of an actor may be created in a scene.
*/
BallActor
/*
The actor name is referenced in the scene file, where actors are instanced.
*/
    {          # start of BallActor
    Graphics
    /*
    The graphics block contains information regarding the display of
    the actor relative to the center of mass of a physics body which it is
    bound to. This relative transformation is specified as a tree of
    scene nodes.
    */
        {
        Node
        # transform node (pos + orient) from group space to
        # body space (COM centered)
        {
        /*
        The top level node has NO transform. It will
        receive the dynamics body's transform that is
        animated.
        */
        Model {BallModel;}
        #all nodes may have an optional model.

        Node
        /*
        All nodes may have unlimited number of child
        nodes.
        */
            {
            /*
            Subnodes may have a transform (pos +
            orient) relative to the parent node.
            (Node2Parent transform)
            */
            position {x;y;z;}
            orientation {w;x;y;z;} #quaternion
            angles {a;b;c;}

```

```

        # used if no orientation is provided
        Model {EraserModel;}
    }
    #more nodes may follow here
}

Collision
{
    /*
    The collision block describes the bounding volumes of the
    actor. This is a list of shapes. The actor will be
    represented as the union of these shapes for collision
    detection. The following shapes are possible in any
    combination and number:
    */
    Box
    {
        position {x;y;z;}
        orientation {w;x;y;z;}
        angles {a;b;c;}
        # used if no orientation is provided
        dimensions {length;width;height;}
    }
    Sphere
    {
        position {x;y;z;}
        radius {r;}
    }
    Capsule
    {
        position {x;y;z;}
        orientation {w;x;y;z;}
        angles {a;b;c;}
        # used if no orientation is provided
        radius {r;}
        height {h;}
    }
    Plane
    {
        normal {x;y;z;}
        d {d;}
    }
    TriangleMesh
    /*
    The meshes specified in the Actors Graphics block
    are used.

```

```

        */
        {
            pmapDensity{a;} #integer
            pmapFile{fileName;} #string
        }
        #more shapes of any kind listed above
    }
}
}

```

3.3 PSC.ODS files

This file describes a scene. Almost all blocks in the file are optional. Here is an example:

```

Scene
{
    Text
    {
        /*
        This block permits text to be displayed during the scene.
        See the source code for the exact specifications.
        */
    }

    PhysStart {1;}          # number of seconds to wait before physics gets activated
    PhysDuration {200;}     # number of seconds to run before going on to next scene
    DragForce {400;}        # force that dragging with the mouse applies
    DragStretch {0.5;}      # maximal distance used for drag force computation
    gravity {0;-9.81;0;}    # the gravity vector in the scene
    fieldOfView {90;}       # angle specifying the field of view
    noGroundPlane;          # deactivates automatic ground plane
    simStepSize {0.02;}     # the size of a physics time step in seconds
    timeStep {0.02;}        # same as simStepSize
    frequency {50;}         # if neither simStepSize nor timeStep is specified, frequency can be used.

    Parameters
    {
        /*
        The term RigidBodySDK and CollisionSDK are obsolete now, since both
        SDK's have been merged into the PhysicsSDK. For downward
        compatibility however, some parameters of the PhysicsSDK are still
        defined in these two blocks. settings have the format:
        PARAMETER {value;}
        see the PhysicsSDK programming API documentation for a description of
        the parameters.

```

```

*/
RigidBodySDK
{
    PENALTY_FORCE {x;}
    MIN_SEPARATION_FOR_PENALTY {x;}
    SLEEP_LIN_VEL_SQUARED {x;}
    SLEEP_ANG_VEL_SQUARED {x;}
    BOUNCE_TRESHOLD {x;}
    DYN_FRICT_SCALING {x;}
    STA_FRICT_SCALING {x;}
    MAX_ANGULAR_VELOCITY {x;}
}
CollisionSDK
{
    MESH_MESH_LEVEL {x;}
    ENABLE_MESH_DEBUG {x;}
    SPHERE_MESH_SMOOTH {x;}
}
}
Bodies
/*
This Block is used to create instances of Actor templates
specified in the pds.ods file.
*/
{
    BallActor #actor name to instance
    {
        /*
        Note that many of the properties shown here are redundant. If you
        specify the mass and the size for the inertia tensor, you don't need
        the density, etc. Please refer to the description of the NxBodyDesc
        class described in the SDK documentation.
        */
        name {tail1;}
        # the name assigned to the body is used by joints and effectors

        # body properties (see NxBodyDesc docu.)
        position {x;y;z;}
        orientation {w;x;y;z;} /*OR*/ angles {x;y;z;}
        scale {s;}
        momentum {x;y;z;}
        angMomentum {x;y;z;}
        mass {m;}
        inertia {it00;it01;it02;it11;it12;it22;}
        #upper triangular elements of symmetric inertia tensor.
        maxAngVel {v;} #maximal angular velocity
    }
}

```

wake {**i**;} #initial interger counter, used for sleep transition.
 linDamping {**d**;} #linear damping
 angDamping {**d**;} #angular damping

#material properties (see NxMaterial docu)
 restitution {**f**;}
 friction {**f**;}
 staticFriction {**f**;}
 frictionV {**f**;}
 staticFrictionV {**f**;}
 dirOfAnisotropy {**x;y;z**;}

#collision detection
 trigger; #selects all Shapes of the Actor to be triggers.
 Collision
 {
 #see description in actor template of the *pds.ods* file.
 }
 suppressDefaultShape; #supresses the generation of a default shape
 dimensions {**halfWidth;halfHeight;halfDepth**;}
 #used for default box shape generation

Control
 {
 /*
 Properties for one of the special case controllers (subtypes
 of the ActController class). Refer to the source code of
 these controllers (e.g. CarController).
 */
 }
}

#more bodies
}

Joints

/*
 A list of joints in the scene. Each joint connects two bodies. Examples
 for some joint types are given below. See the SDK documentation about
 the meaning of the joint parameters. angle values are given in degrees.
 */
 {
 Spherical
 {
 name {**jointName**;}
 anchor {**x;y;z**;}
 direction {**x;y;z**;}
 twistLowLimit {**value;restitution;hardness**;}
 }
 }
}

```

twistHighLimit {value;restitution;hardness;}
swingLimit {value;restitution;hardness;}
swingAxis {x;y;z;}
swingSpring {spring;damper;targetValue;}
twistSpring {spring;damper;targetValue;}
bodies {namebody1;namebody2;}
#one of these body names may be 'WORLD'.
}
Hinge
/*
The name of this joint has changed from "Hinge-Joint" to
"Revolute Joint" in the SDK.
*/
{
name {jointName;}
anchor {x;y;z;}
direction {x;y;z;}
lowLimit {value;restitution;hardness;}
highLimit {value;restitution;hardness;}
motor {velTarget;maxForce;freeSpin;}
springDamper {spring;damper;targetValue;}
bodies {namebody1;namebody2;}
#one of these body names may be 'WORLD'.
}
PointOnLine
{
name {jointName;}
anchor {x;y;z;}
direction {x;y;z;}
bodies {LINEbody1;POINTbody2;}
# one of these body names may be 'WORLD'.
# note how body order is significant in this
# asymmetric joint.
}
PointInPlane
{
name {jointName;}
anchor {x;y;z;}
normal {x;y;z;}
bodies {PLANE;POINTbody2;}
#one of these body names may be 'WORLD'.
# note how body order is significant in this
# asymmetric joint.
}
CollisionVeto
/*

```

A special joint type not in the SDK that prevents two bodies from reporting collisions.

```
*/
{
  bodies {namebody1;namebody2;}
  #one of these body names may be 'WORLD'.
}
```

Sliding

```
{
  name {jointName;}
  anchor {x;y;z;}
  direction {x;y;z;}
  bodies {namebody1;namebody2;}
  #one of these body names may be 'WORLD'.
}
```

Prismatic

```
{
  name {jointName;}
  anchor {x;y;z;}
  direction {x;y;z;}
  bodies {namebody1;namebody2;}
  #one of these body names may be 'WORLD'.
```

*# This last joint example shows the setting of limits
as described in the SDK manual, limits work the same for all
joint types*

```
limitPoint_2 {x;y;z;}
#could also be limitPoint_1, to attach to body1.
limitPlanes
{
  # a list of one or more limit planes
  planeName # unused
  {
    n {x;y;z;} # plane normal
    p {x;y;z;} # a point in the plane
  }
  plane2Name
  {
    n {x;y;z;} # plane normal
    p {x;y;z;} # a point in the plane
  }
  # ...more planes
}
```

```
}
}
```

3.4 APP.GLA.ODS file

The graphics framework of the viewer loads this file. It has the following format:

```
appInfo
{
  glutGameModeString {"1024x768:32";}
  dumpGLInfo {0;}
}
```

When this file is not found, or no glutGameModeString is specified, the viewer runs in windowed mode. Otherwise, the full screen resolution specified is selected.

3.5 MOD.ODS files

The Model file basically assigns materials to the different face groups in a model.

```
Model3
{
  Mesh {"eraser.obj";} # the mesh file
  Scale {s;}
  TexCoords {1;} # one or zero
  ForceNormals {1;} # one or zero
  ForceTexCoords {1;} # one or zero
  Groups
  {
    # for each group name given in the mesh file,
    # an entry here should be specified.
    Default {Eraser.mat.ods;} # a material file
    bottom {Eraser.mat.ods;}
    top {Eraser.mat.ods;}
  }
}
```

3.6 MAT.ODS files

This material file is very simple:

```
Material3
{
  Smooth {0;} # smooth or flat shading.
  Color {r;g;b;a;}
  Specular {r;g;b;a;}
  SpecExp {0.100000;} # specular exponent
  Tex1 {"street-right-top.bmp";} # texture map. Various 24-bit file formats.
}
```

3.7 OBJ files

This is the standard Alias|Wavefront object file. See publicly available documentation on the web.